

RECON//REPORT

ReconToReport: URL → Vulnerability Report

An automated, authorization-gated web-application security pipeline

Reconnaissance · Traffic Capture · Scanning · Exploitation · Reporting

BACKGROUND

- Web-app testing requires orchestrating many **disconnected open-source tools**, each with its own output format.
- Analysts spend hours manually correlating results and **hand-writing reports**.
- Ad-hoc scanning risks going **out of scope** — a legal and ethical hazard.

PURPOSE

Build one framework that takes a single target URL plus explicit authorization and produces a structured, client-ready vulnerability report — with every active probe gated behind consent.

METHODOLOGY

System design

A Python orchestrator sequences five phases, each normalizing its tools' output into one shared SQLite data model (targets, assets, transactions, findings, evidence, secrets, auth contexts).

Phased pipeline

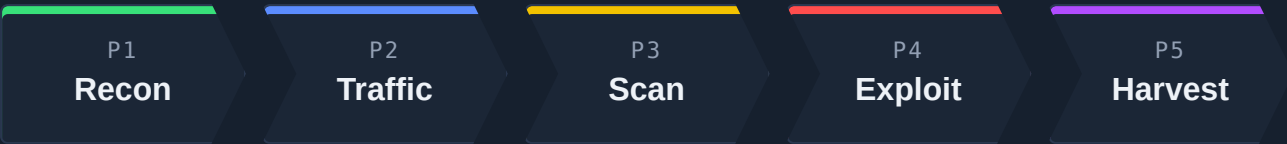


Fig 1. Phases declare dependencies; selecting one auto-runs its prerequisites. A failure in any phase is isolated so the run continues.

Key mechanisms

- Authorization gate** — active phases refuse to run without explicit consent.
- Built-in crawler** — maps attack surface with zero external tools.
- Bounded execution** — per-tool time budgets prevent runaway scans.

DISCUSSION

Normalizing heterogeneous tool output into a single schema makes findings from nuclei, sqlmap, JWT analysis, and the secret scraper directly comparable and the whole run reproducible.

Because discovery feeds scanning feeds exploitation, later phases act on concrete, in-scope targets rather than guesses — improving signal and keeping traffic within authorization.

RESULTS

- Live run against a public, intentionally-vulnerable target (Google Firing Range): **326 assets discovered, 20 reflected/DOM-XSS findings**.
- On a controlled target: a **critical “JWT alg:none”** finding and multiple secret matches, each with CVSS, impact, and remediation.



Severity distribution

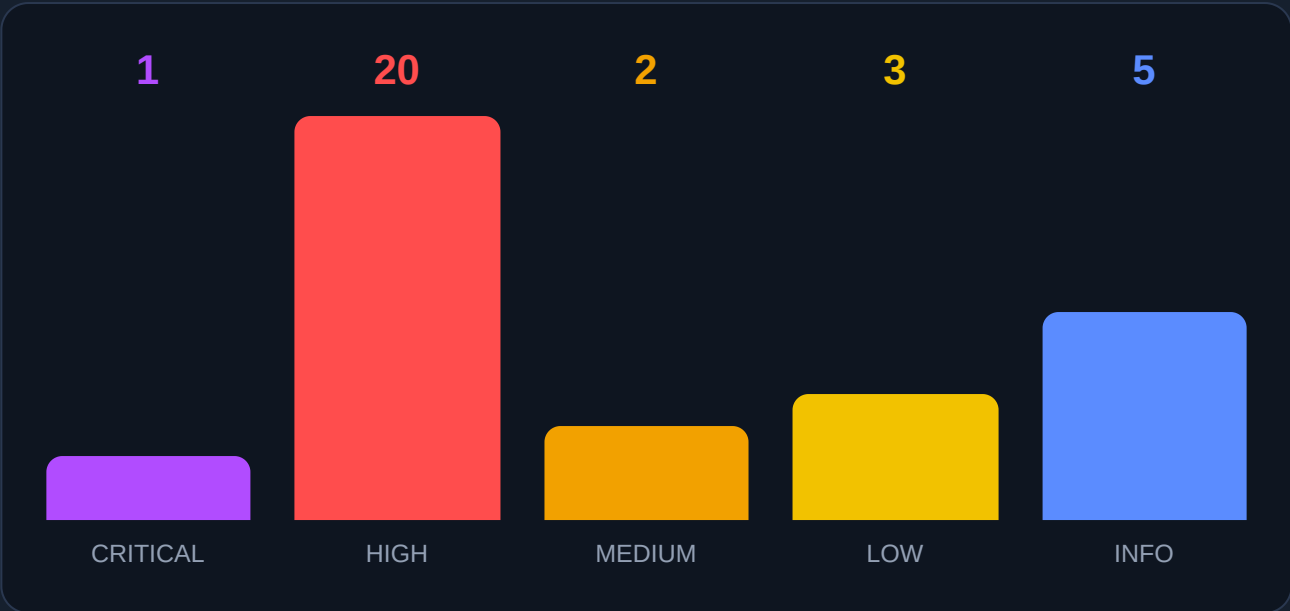


Fig 2. Findings grouped by severity across a representative run.

Sample findings

FINDING	SEVERITY	CWE
JWT accepts alg:none (unsigned)	CRITICAL	CWE-347
Reflected XSS via query parameter	HIGH	CWE-79
Broken access control (privilege diff)	HIGH	CWE-285
Sensitive file exposed (.git/.env)	MEDIUM	CWE-538

Table 1. Representative findings with CVSS, impact, and remediation in the full report.

CONCLUSIONS

ReconToReport collapses a fragmented manual workflow into one reproducible, consent-first pipeline: supply a URL and authorization, receive a severity-ranked report and a live dashboard.

- End-to-end automation from discovery to rendered report.
- Comparable, correlated findings via a single data model.
- Safety by design — no active testing without authorization.

FUTURE WORK

- Active parameter fuzzing for deeper injection coverage
- Out-of-band (interactsh) confirmation for blind flaws
- Host liveness filtering before scanning · PDF/CSV export

TECH STACK

Python

SQLAlchemy

Alembic

Flask

Jinja2

mitmproxy

Playwright

nuclei

sqlmap

FOR MORE INFORMATION

Project: **ReconToReport**
Launch: **r2r ui**
Scope: authorized security testing only